

数据结构

第二章 线性表

线性表的类型定义

线性表的顺序表示和实现

线性表的链式表示和实现

一元多项式的表示及相加

第二章 线性表

线性结构是最常用、最简单的一种数据结构。而线性表是一种典型的线性结构。其基本特点是线性表中的数据元素是有序且是有限的。

线性结构**特点**：在数据元素的非空有限集中

- 存在**唯一**的一个被称作“**第一个**”的数据元素
- 存在**唯一**的一个被称作“**最后一个**”的数据元素
- 除第一个外，集合中的每个数据元素均**只有一个前驱**
- 除最后一个外，集合中的每个数据元素均**只有一个后继**

线性表的种类

链表、栈、队列、串、数组

2.1 线性表的类型定义

线性表(**Linear List**)：是由 $n(n \geq 0)$ 个数据元素(结点) $a_1, a_2, \dots, a_n$ 组成的有限序列。该序列中的所有结点具有相同的数据类型。其中数据元素的个数 n 称为线性表的长度。

➤ 当 $n=0$ 时，称为空表。

➤ 当 $n>0$ 时，将非空的线性表记作： $(a_1, a_2, \dots, a_n)$

a_1 称为线性表的第一个(首)结点， a_n 称为线性表的最后一个(尾)结点。

$a_1, a_2, \dots, a_{i-1}$ 都是 $a_i (2 \leq i \leq n)$ 的前驱，其中 a_{i-1} 是 a_i 的直接前驱；

$a_{i+1}, a_{i+2}, \dots, a_n$ 都是 $a_i (1 \leq i \leq n-1)$ 的后继，其中 a_{i+1} 是 a_i 的直接后继。

线性表的逻辑结构

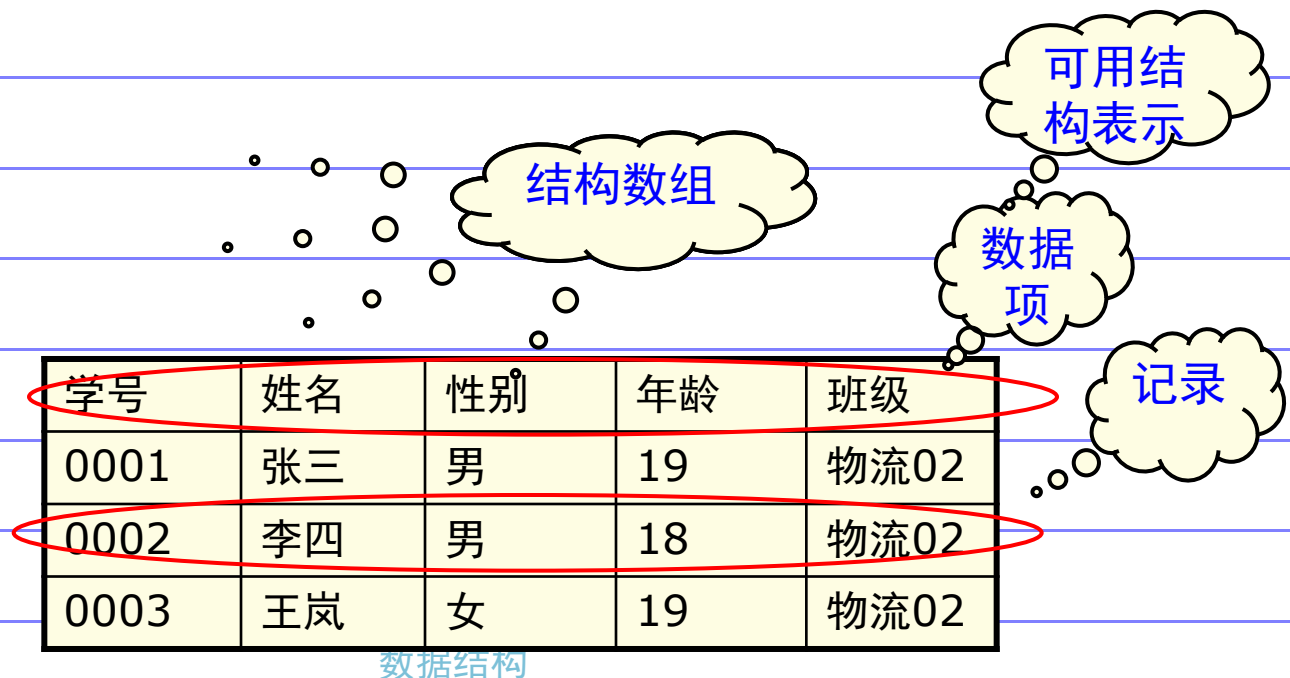
线性表中的数据元素 a_i 所代表的具体含义随具体应用的不同而不同，在线性表的定义中，只不过是一个抽象的表示符号。

◆ 线性表中的结点可以是单值元素(每个元素只有一个数据项)。

- 例1: 26个英文字母组成的字母表: (A, B, C, ..., Z)
- 例2: 某校从1978年到1983年各种型号的计算机拥有量的变化情况: (6, 17, 28, 50, 92, 188)
- 例3: 一副扑克的点数 (2, 3, 4, ..., J, Q, K, A)

线性表的逻辑结构

◆ 线性表中的结点可以是记录型元素，每个元素含有多个数据项，每个项称为结点的一个域。每个元素有一个可以唯一标识每个结点的数据项组，称为关键字。



□ 要求：

- 元素同构，属于同一数据对象。通常可用C语言的数据类型描述
- 不能出现缺项

线性表的形式定义

□ 定义：含有n个元素的线性表是一个数据结构

$$\text{Linear_List} = (D, R)$$

其中， $D = \{a_i | a_i \in \text{ElemSet}, i = 1, 2, \dots, n \geq 0\}$

$$R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 1, 2, \dots, n \}$$

□ 元素个数n — 称为表的长度，当n=0称为空表

□ 当 $1 < i < n$ 时

■ a_i 的直接前驱是 a_{i-1} ， a_1 无直接前驱

■ a_i 的直接后继是 a_{i+1} ， a_n 无直接后继

□ 若线性表中的结点是按值(或按关键字值)由小到大(或由大到小)排列的，称线性表是有序的。

□ 线性表是一个相当灵活的数据结构，它的长度因需可长可短。因此对线性表的操作不仅有查询，还有插入和删除。

线性表的抽象数据类型定义

□ ADT List{

- 数据对象: $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$
- 数据关系: $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$
- 基本操作:
- **InitList(&L)**
- 操作结果: 构造一个空的线性表L;
- **ListLength(L)**
- 初始条件: 线性表L已存在;
- 操作结果: 若L为空表, 则返回TRUE, 否则返回FALSE;
-
- **GetElem(L, i, &e)**
- 初始条件: 线性表L已存在, $1 \leq i \leq \text{ListLength}(L)$;
- 操作结果: 用e返回L中第i个数据元素的值;
- **ListInsert (& L, i, e)**
- 初始条件: 线性表L已存在, $1 \leq i \leq \text{ListLength}(L)$;
- 操作结果: 在线性表L中的第i个位置插入元素e;

□ ...

□ } ADT List

线性表的抽象数据类型定义

- ❑ InitList(&L)
- ❑ DestroyList(&L)
- ❑ ClearList(&L)
- ❑ ListEmpty(L)
- ❑ ListLength(L)
- ❑ GetElem(L, I, &e)
- ❑ LocateElem(L, e, compare())
- ❑ PriorElem(L, cur_e, &pre_e)
- ❑ NextElem(L, cur_e, &next_e)
- ❑ ListInsert(&L, i, e)
- ❑ ListDelete(&L, i, &e)
- ❑ ListTraverse(L, visit())
- ❑ 对于上述定义的抽象数据类型线性表，还可进行一些更复杂的操作，例如：将两个或两个以上的线性表合成一个线性表；把一个线性表拆分成两个或两个以上的线性表；重新复制一个线性表；

线性表的扩展操作

□ 例:线性表La和Lb表示两个集合, 现要求两集合的并, 并存放在线性表La中

```
void union(List& La, List Lb) {  
    // 将所有在线性表Lb中但不在La中的数据元素插入到La中  
    // 分析: 显然, 要求La和Lb的元素是同类数据对象。  
    La_len = ListLength(La); Lb_len = ListLength(Lb);  
    for(i = 1; i <= Lb_len; i++)  
    {  
        GetElem(Lb, i, e);  
        if(!LocateElem(La, e, equal))  
            ListInsert(La, ++ La_len, e);  
    }  
}
```

线性表的扩展操作

□ 例:线性表La和Lb是按元素序非递减的, 合并两个线性表为Lc, 使Lc的元素亦为非递减序

若Lc为空表, 仅需考虑La和Lb元素。用两个指示变量分别取La和Lb的元素, 比较其大小, 依次加入到Lc中。(1) 若Lc不为空? (2) La、Lb无序呢?

```
void MergeList(List La, List Lb, List Lc) {  
    InitList(Lc);  
    i = j = 1; k = 0;  
    La_len = ListLength(La); Lb_len = ListLength(Lb);  
    while(i <= La_len && j <= Lb_len) {  
        GetElem(La, i, ai); GetElem(Lb, j, bj);  
        if( ai <= bj)  
            { ListInsert(Lc, ++k, ai); i++; }  
        else  
            { ListInsert(Lc, ++k, bj); j++; }  
    }  
    while(i <= La_len) {  
        GetElem(La, i++, ai); ListInsert(Lc, ++k, ai);  
    }  
    while(j <= Lb_len) {  
        GetElem(Lb, j++, bj); ListInsert(Lc, ++k, bj);  
    }  
}
```

2.2 线性表的顺序表示和实现

顺序存储：把线性表的结点**按逻辑顺序**依次存放在一组地址连续的存储单元里。用这种方法存储的线性表简称顺序表。

顺序存储的线性表的特点：

- ◆ 线性表的逻辑顺序与物理顺序一致；
- ◆ 数据元素之间的关系是以元素在计算机内“物理位置相邻”来体现。

设有非空的线性表： $(a_1, a_2, \dots, a_n)$ 。顺序存储如图2-1所示。

2.2 线性表的顺序表示和实现

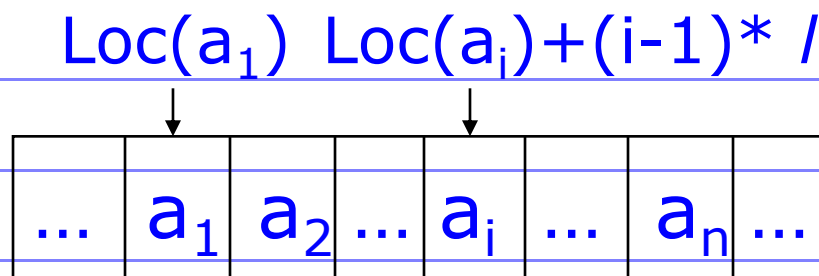


图2-1 线性表的顺序存储表示

- 元素地址计算方法(以所占的第一个单元的存储地址作为数据元素的存储位置)
 - $\text{LOC}(a_{i+1}) = \text{LOC}(a_i) + L$
 - $\text{LOC}(a_i) = \text{LOC}(a_1) + (i-1) * L$
- 其中:
 - L — 一个元素占用的存储单元个数
 - $\text{LOC}(a_i)$ — 线性表第 i 个元素的地址
- $\text{LOC}(a_1)$ 是线性表第一个元素 a_1 的存储地址, 也叫线性表的起始位置或基地址
- 线性表的这种内存存储表示也叫线性表的顺序存储结构, 或顺序映像(Sequential Mapping)

顺序表的内存映像

□ 特点:

■ 实现逻辑上相邻—物理地址相邻

■ 实现随机存取

内存地址	存储内容	元素序号	数组下标
b	a_1	1	0
b+l	a_2	2	1
	...		
$b+(n-1)*l$	a_n	n	n-1
			N-1

顺序表的内存映像

□ 实现：可用C语言的一维数组实现

在高级语言(如C语言)环境下：数组具有随机存取的特性，因此，借助数组来描述顺序表。除了用数组来存储线性表的元素之外，顺序表还应该有表示线性表的长度属性，所以用结构类型来定义顺序表类型。

```
#define OK 1
```

```
#define ERROR -1
```

```
#define MAX_SIZE 100
```

```
#define LIST_INIT_SIZE 100
```

```
typedef int Status ;
```

```
typedef int ElemType ;
```

```
typedef struct sqlist
```

```
{ ElemType Elem_array[MAX_SIZE] ;
```

```
int length ; int Listsize;
```

```
} SqList ;
```

```
typedef struct sqlist
```

```
{ ElemType *Elem ;
```

```
int length ; int Listsize;
```

```
} SqList ;
```


顺序表基本操作

☐ 关于数据元素的定义



☐ 初始化



☐ 撤销



☐ 插入一个元素



☐ 删除一个元素



☐ 插入/删除算法的复杂性



☐ 查找/定位一个元素



☐ 顺序表的特点



关于数据元素的定义

□ 在前面的例子中

$(a_1, a_2, a_3, \dots, a_n)$

$(A, B, C, \dots, Z)$

$(6, 17, 28, 50, 92, 188)$



学号	姓名	性别	年龄	班级
0001	张三	男	19	物流02
0002	李四	男	18	物流02
0003	王岚	女	19	物流02

顺序表初始化操作

```
Status InitList_Sq(SqList &L)
```

```
{
```

```
    L.Elem =
```

```
        (ElemType*)malloc(LIST_INIT_SIZE*sizeof(ElemType));
```

```
    if(!L.elem) exit(OVERFLOW);
```

```
    L.Length = 0;
```

```
    L.Listsize = LIST_INIT_SIZE;
```

```
    return OK;
```

```
}
```

顺序表插入操作

在线性表 $L = (a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 中的第 i ($1 \leq i \leq n$) 个位置上插入一个新结点 e , 使其成为线性表:

$$L = (a_1, \dots, a_{i-1}, e, a_i, a_{i+1}, \dots, a_n)$$

实现步骤

- (1) 将线性表 L 中的第 i 个至第 n 个结点后移一个位置。
- (2) 将结点 e 插入到结点 a_{i-1} 之后。
- (3) 线性表长度加 1。

顺序表插入操作

□ 在顺序表中指定的位置插入

■ 如果对于定长的表, 已经规定了表的长度

✧ 根据线性表的定义, 要保证位置相邻性, 必须移动其后的数据元素, 移动次数为 $(n-i+1)$, 移动操作的平均次数为 $n/2$

■ 如果非指定长度, 需要增加存储单元, 然后按上述操作

✧ 当然平均移动次仍为 $n/2$

□ 在顺序表中末尾插入(追加AppendElem)

■ 无需移动, 如果没有指定长度, 则需增加存储单元

顺序表插入操作

算法描述

Status Insert_SqList(SqList *L, int i , ElemType e)

{ int j ;

if (i<1||i>L->length+1) return ERROR ;

if (L->length>=MAX_SIZE)

{ printf(“线性表溢出!\n”); return ERROR ; }

for (j=L->length-1; j>=i-1; --j)

L->Elem_array[j+1]=L->Elem_array[j];

/* i-1位置以后的所有结点后移 */

L->Elem_array[i-1]=e; /* 在i-1位置插入结点 */

L->length++ ;

return OK ;

}

顺序表插入操作

时间复杂度分析

在线性表L中的第i个元素之前插入新结点，其时间主要耗费在表中结点的移动操作上，因此，可用结点的移动来估计算法的时间复杂度。

设在线性表L中的第i个元素之前插入结点的概率为 P_i ，不失一般性，设各个位置插入是等概率，则 $P_i = 1/(n+1)$ ，而插入时移动结点的次数为 $n-i+1$ 。

总的平均移动次数： $E_{\text{insert}} = \sum p_i * (n-i+1) \quad (1 \leq i \leq n)$

$\therefore E_{\text{insert}} = n/2$ 。

即在顺序表上做插入运算，平均要移动表上一半结点。当表长n较大时，算法的效率相当低。因此算法的平均时间复杂度为 $O(n)$ 。

顺序表插入操作

```
Status InsertList (SqList &L, int i, ElemType e) {  
    // 在顺序线性表 L 中第 i 个位置之前插入新的元素 e,  
    // i 的合法值为  $1 \leq i \leq \text{ListLength\_Sq}(L) + 1$   
    if (i < 1 || i > L.length + 1) return ERROR; // i 值不合法  
    if (L.length >= L.listsize) { // 当前存储空间已满, 增加分配  
        newbase = (ElemType *)realloc(L.elem,  
            (L.listsize + LISTINCREMENT) * sizeof (ElemType));  
        if (!newbase) exit(OVERFLOW); // 存储分配失败  
        L.elem = newbase; // 新基址  
        L.listsize += LISTINCREMENT; // 增加存储容量  
    }  
    q = &(L.elem[i - 1]); // q 为插入位置  
  
    for (p = &(L.elem[L.length - 1]); p >= q; --p)  
        *(p + 1) = *p; // 插入位置及之后的元素右移  
    *q = e; // 插入 e  
    ++L.length; // 表长增 1  
    return OK;  
} // InsertList
```

再次注意这里:
给定了表长度,
若表长度不够,
仍重分配内存

顺序表删除操作

在线性表 $L=(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 中删除结点 a_i ($1 \leq i \leq n$), 使其成为线性表:

$$L = (a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$$

实现步骤

- (1) 将线性表 L 中的第 $i+1$ 个至第 n 个结点依此向前移动一个位置。
- (2) 线性表长度减 1。

顺序表删除操作

算法描述

```
ElemType Delete_SqList(SqList *L, int i)
{
    int k; ElemType x;
    if (L->length==0)
        { printf("线性表L为空!\n"); return ERROR; }
    else if ( i<1||i>L->length )
        { printf("要删除的数据元素不存在!\n");
          return ERROR ; }
    else { x=L->Elem_array[i-1]; /*保存结点的值*/
          for ( k=i; k<L->length; k++)
              L->Elem_array[k-1]=L->Elem_array[k];
              /* i位置以后的所有结点前移 */
          L->length--; return (x);
        }
}
```

顺序表删除操作

时间复杂度分析

删除线性表L中的第i个元素，其时间主要耗费在表中结点的移动操作上，因此，可用结点的移动来估计算法的时间复杂度。

设在线性表L中删除第i个元素的概率为 P_i ，不失一般性，设删除各个位置是等概率，则 $P_i = 1/n$ ，而删除时移动结点的次数为 $n-i$ 。

则总的平均移动次数： $E_{\text{delete}} = \sum p_i * (n-i) \quad (1 \leq i \leq n)$

$\therefore E_{\text{delete}} = (n-1)/2$ 。

即在顺序表上做删除运算，平均要移动表上一半结点。当表长n较大时，算法的效率相当低。因此算法的平均时间复杂度为 $O(n)$ 。

顺序表查找操作

- 给出一个位置 i ($1 \leq i \leq n$), 直接定位
- 给出一个元素 e

```
int LocateElem_Sql(SqlList L, ElemType e,
    Status (*compare)(ElemType, ElemType) {
    // 在顺序表L中查找第1个值与e满足
    // compare()的元素的位置, 若找到,
    // 则返回其在L中的位置, 否则返回0
    i = 1;          // i 的初值为第1个元素的位置
    p = L.elem;     // p 的初值为第1个元素的存储位置
    while (i <= L.length && !(*compare)(*p++, e))
        ++i;
    if (i <= L.length) return i;
    return 0;
} // LocateElem_Sql
```

顺序线性表的查找定位删除

在线性表 $L = (a_1, a_2, \dots, a_n)$ 中删除值为 x 的第一个结点。

实现步骤

- (1) 在线性表 L 查找值为 x 的第一个数据元素。
- (2) 将从找到的位置至最后一个结点依次向前移动一个位置。
- (3) 线性表长度减1。

顺序线性表的查找定位删除

算法描述

Status Locate_Delete_SqList(SqList *L, ElemType x)

/* 删除线性表L中值为x的第一个结点 */

{ int i=0, k, m = L->length;

while (i<L->length) /*查找值为x的第一个结点*/

{ if (L->Elem_array[i]!=x) i++;

else

{ for (k=i+1; k< L->length; k++)

L->Elem_array[k-1] = L->Elem_array[k];

L->length--; break ;

}

}

if (i ≥ m)

{ printf(“要删除的数据元素不存在!\n”);

return ERROR ; }

return OK;

}

顺序线性表的查找定位删除

时间复杂度分析

时间主要耗费在数据元素的比较和移动操作上。

首先，在线性表L中查找值为x的结点是否存在；

其次，若值为x的结点存在，且在线性表L中的位置为i，则在线性表L中删除第i个元素。

设在线性表L删除数据元素概率为 P_i ，不失一般性，设各个位置是等概率，则 $P_i = 1/n$ 。

◆ 比较的平均次数： $E_{\text{compare}} = \sum p_i * i \quad (1 \leq i \leq n)$

∴ $E_{\text{compare}} = (n+1)/2$ 。

◆ 删除时平均移动次数： $E_{\text{delete}} = \sum p_i * (n-i) \quad (1 \leq i \leq n)$

∴ $E_{\text{delete}} = (n-1)/2$ 。 平均时间复杂度：

$E_{\text{compare}} + E_{\text{delete}} = n$ ，即为 $O(n)$

顺序存储结构的优缺点

□ 优点

- 逻辑相邻，物理相邻
- 可随机存取任一元素
- 存储空间使用紧凑

□ 缺点

- 插入、删除操作需要移动大量的元素
- 预先分配空间需按最大空间分配，利用不充分
- 表容量难以扩充

线性表的链式表示和实现

链式存储：用一组任意的存储单元存储线性表中的数据元素。用这种方法存储的线性表简称**线性链表**。

存储链表中结点的一组任意的存储单元可以是连续的，也可以是不连续的，甚至是零散分布在内存中的任意位置上的。

链表中结点的逻辑顺序和物理顺序不一定相同。

线性表的链式表示和实现

- ❑ 为了正确表示结点间的逻辑关系，在存储每个结点值的同时，还必须存储指示其直接后继结点的地址(或位置)，称为指针(pointer)或链(link)，这两部分组成了链表中的结点结构，如下图所示。
- ❑ 链表是通过每个结点的指针域将线性表的 n 个结点按其逻辑次序链接在一起的。
- ❑ 每一个结只包含一个指针域的链表，称为单链表。
- ❑ 为操作方便，总是在链表的第一个结点之前附设一个头结点(头指针)head指向第一个结点。头结点的数据域可以不存储任何信息(或链表长度等信息)。

data	next
-------------	-------------

data : 数据域，存放结点的值。

next : 指针域，存放结点的直接后继的地址。

线性表的链式表示和实现

□ 特点:

- 用一组任意存储单元存储线性表的数据元素
- 利用指针实现了用不相邻的存储单元存放逻辑上相邻的元素
- 每个数据元素 a_i ，除存储本身信息外，还需存储其直接后继的地址(或位置)——结点
 - ◇ 数据域：元素本身信息
 - ◇ 指针域：指示直接后继的存储位置(链)

结点

数据域	指针域
-----	-----

```
struct Linkage {  
    DataType *Data;  
    struct Linkage *Next;  
};
```

线性表的链式表示例

例：线性表

(ZHAO, QIAN, SUN,
LI, ZHOU, WU,
ZHENG, WANG)

存储地址

1

7

13

19

25

31

37

43

数据域

指针域

LI

43

QIAN

13

SUN

1

WANG

NULL

WU

37

ZHAO

7

ZHENG

19

ZHOU

25

头指针

H

31

ZHAO

QIAN

SUN

LI

ZHOU

WU

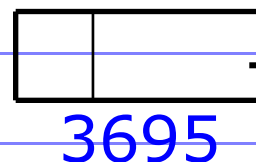
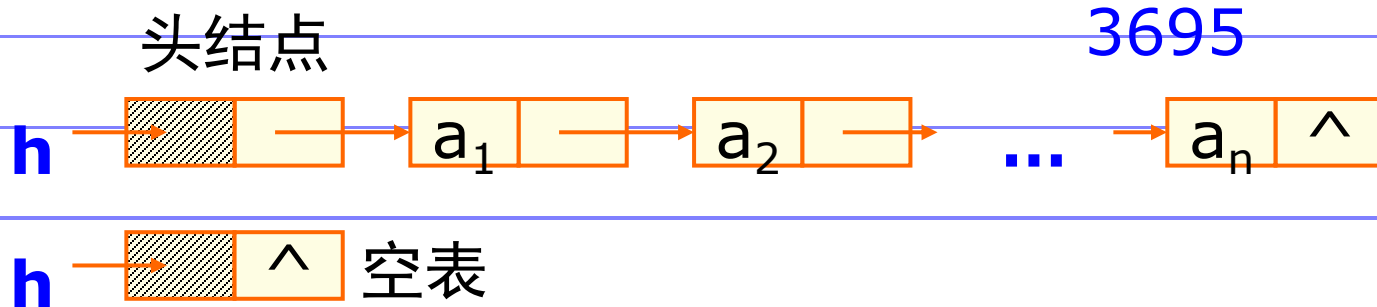
ZHENG

WANG

^

头结点

- 为操作方便，总是在链表的第一个结点之前附设一个头结点(头指针) **head** 指向第一个结点。头结点的数据域可以不存储任何信息(或链表长度等信息)
- 单链表是由表头唯一确定，因此单链表可以用头指针的名字来命名。
- 头结点指针域为空表示线性表为空
- 在单链表中，任何两个元素的存储位置之间没有固定的关系。然而，每个元素的存储位置都包含在其直接前驱结点的信息之中。



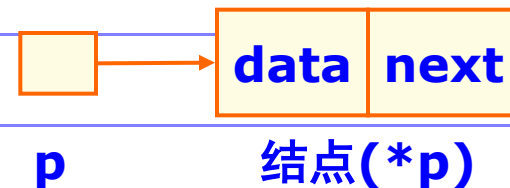
	
1100	hat
	NULL
	
1300	cat
	1305
1305	eat
	3700
	
→	bat
	1300
	fat
	1100
	

线性链表-结点的描述

□ 定义：结点中只含一个指针域的链表称为单链表

□ 实现：

```
typedef struct LNode {  
    ElemType data;  
    struct LNode *next;  
} LNode, *LinkList;  
LNode *h,*p;
```



(***p**)表示**p**所指向的结点

(***p**).data $\Leftrightarrow$ p->data表示**p**指向结点的数据域

(***p**).next $\Leftrightarrow$ p->next表示**p**指向结点的指针域

线性链表-结点的实现

结点是通过动态分配和释放来的实现，即需要时分配，不需要时释放。实现时是分别使用C语言提供的标准函数：

malloc() , **realloc()** , **sizeof()** , **free()** 。

动态分配 **p=(LNode*)malloc(sizeof(LNode));**

函数**malloc**分配了一个类型为**LNode**的结点变量的空间，并将其首地址放入指针变量**p**中。

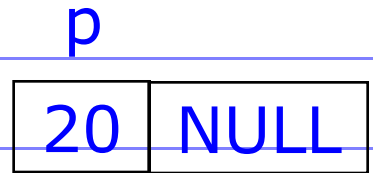
动态释放 **free(p);**

系统回收由指针变量**p**所指向的内存区。**P**必须是最近一次调用**malloc**函数时的返回值。

线性链表-基本操作

(1) 结点的赋值

■ **LNode *p;**

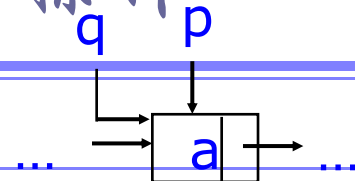
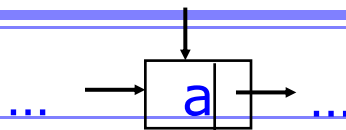


■ **p=(LNode*)malloc(sizeof(LNode));**

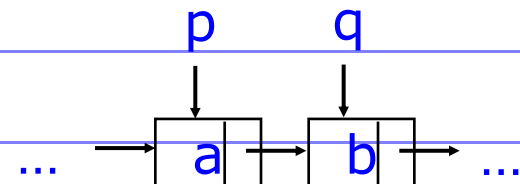
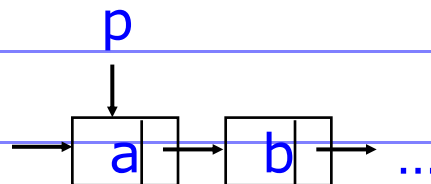
■ **p->data=20; p->next=NULL ;**

线性链表-常见的指针操作

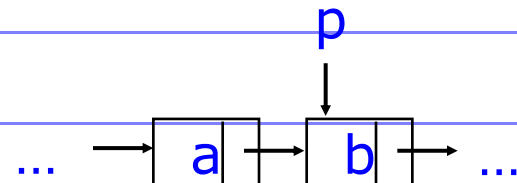
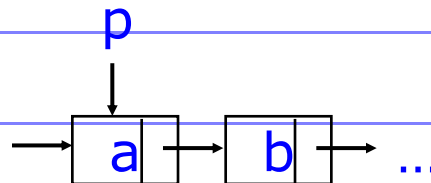
① $q = p ;$



② $q = p \rightarrow \text{next} ; \dots$

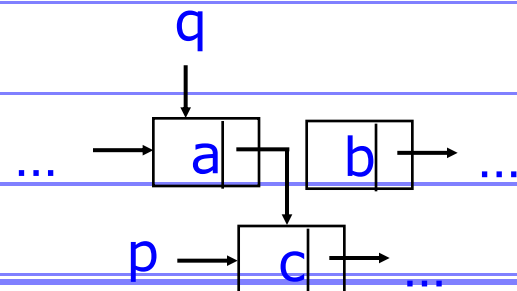
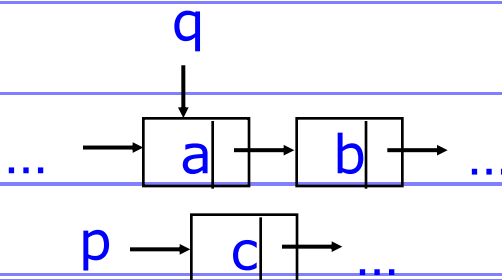


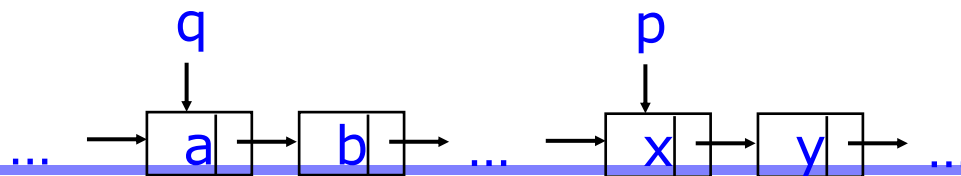
③ $p = p \rightarrow \text{next} ; \dots$



④ $q \rightarrow \text{next} = p ;$

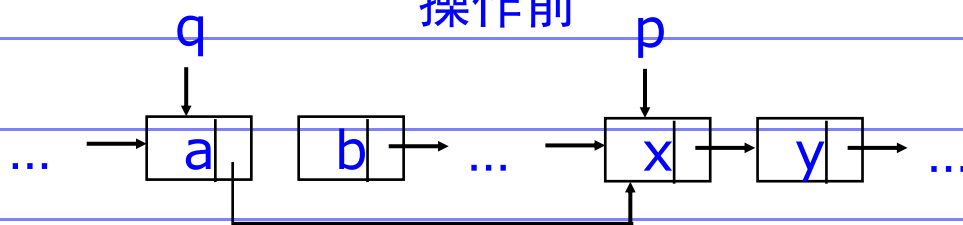
(a)





(b)

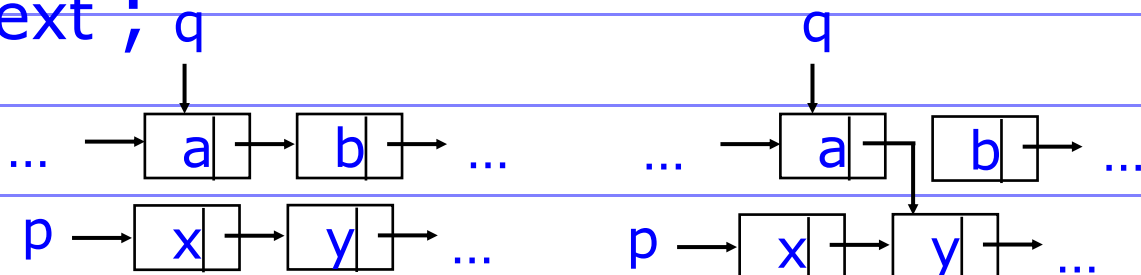
操作前



操作后

⑤ $q \rightarrow \text{next} = p \rightarrow \text{next};$ q

(a)

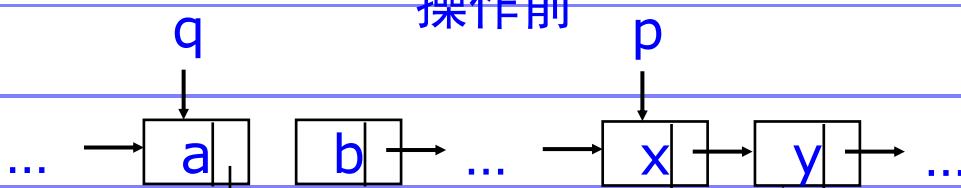


操作前

操作后

(b)

操作前



操作后

数据结构

单链表的动态建立

假设线性表中结点的数据类型是整型，以32767作为结束标志。动态地建立单链表的常用方法有如下两种：头插入法，尾插入法。

(1) 头插入法建表

从一个空表开始，重复读入数据，生成新结点，将读入数据存放到新结点的数据域中，然后将新结点插入到当前链表的表头上，直到读入结束标志为止。即每次插入的结点都作为链表的第一个结点。

单链表的动态建立

算法描述

```
LNode *create_LinkList(void)
```

```
/* 头插入法创建单链表,链表的头结点head作为返回值 */
```

```
{ int data ;
```

```
  LNode *head, *p;
```

```
  head= (LNode *) malloc( sizeof(LNode));
```

```
  head->next=NULL;      /* 创建链表的表头结点head */
```

```
  while (1)
```

```
  { scanf("%d", &data) ;
```

```
    if (data==32767) break ;
```

```
    p= (LNode *)malloc(sizeof(LNode));
```

```
    p->data=data;      /* 数据域赋值 */
```

```
    p->next=head->next ; head->next=p ;
```

```
    /* 钩链,新创建的结点总是作为第一个结点 */
```

```
  }
```

```
  return (head);
```

```
}
```


单链表的动态建立

(2) 尾插入法建表

头插入法建立链表虽然算法简单，但生成的链表中结点的次序和输入的顺序相反。若希望二者次序一致，可采用尾插法建表。该方法是将新结点插入到当前链表的表尾，使其成为当前链表的尾结点。

单链表的动态建立

```
LNode *create_LinkList(void)
```

```
/* 尾插入法创建单链表,链表的头结点head作为返回值 */
```

```
{ int data ;
```

```
    LNode *head, *p, *q;
```

```
    head=p=(LNode *)malloc(sizeof(LNode));
```

```
    p->next=NULL;      /* 创建单链表的表头结点head */
```

```
    while (1)
```

```
    {   scanf("%d",& data);
```

```
        if (data==32767) break ;
```

```
        q= (LNode *)malloc(sizeof(LNode));
```

```
        q->data=data;    /* 数据域赋值 */
```

```
        q->next=p->next; p->next=q; p=q ;
```

```
        /*钩链, 新创建的结点总是作为最后一个结点*/
```

```
    }
```

```
    return (head);
```

```
}
```

无论是哪种插入方法,如果要插入建立的单线性链表的结点是n个,算法的时间复杂度均为 $O(n)$ 。

单链表中查找与定位元素

(1) 按序号查找 取单链表中的第 i 个元素。

对于单链表，不能象顺序表中那样直接按序号 i 访问结点，而只能从链表的头结点出发，沿链域`next`逐个结点往下搜索，直到搜索到第 i 个结点为止。因此，链表不是随机存取结构。

设单链表的长度为 n ，要查找表中第 i 个结点，仅当 $1 \leq i \leq n$ 时， i 的值是合法的。

```
❑ Status GetElem_L(LinkList L, int i, ElemType &e){  
    p = L->next; j = 1;  
    while (p && j<i) {  
        p = p->next; ++j;  
    }  
    if (!p || j>i) return ERROR;  
    e = p->data;  
    return OK;  
} // GetElem_L
```

移动指针p的频率:

❑ 算法评价: $i < 1$ 时: 0次; $i \in [1, n]$: $i-1$ 次; $i > n$: n 次。

时间复杂性 $T(n) = O(n)$

单链表中查找与定位元素

(2) 按值查找

按值查找是在链表中，查找是否有结点值等于给定值 **key** 的结点？若有，则返回首次找到的值为 **key** 的结点的存储位置；否则返回 **NULL**。查找时从开始结点出发，沿链表逐个将结点的值和给定值 **key** 作比较。

单链表中查找与定位元素

- 查找：查找单链表中是否存在数据为X的结点，若有则返回指向含X的结点指针；否则返回NULL。算法描述如下：

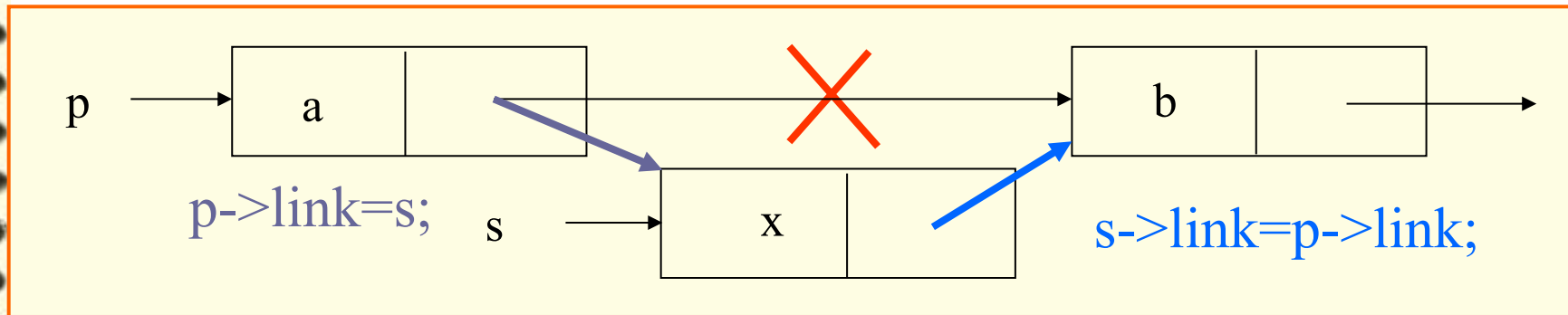
```
LinkedList *LinkedListSearch(LinkedList L, ElemType X){  
    LinkedList *p = L->next;  
    while(p!=NULL && p->data!=X)  
        p=p->next;  
    return (p);  
}
```

- 算法评价：

- 最好情况首节点即是，最坏情况是未找到，平均循环和比较次数为 $(n+1)/2$.
- 时间复杂性 $T(n) = O(n)$

单链表中插入结点

插入运算是将值为 e 的新结点插入到表的第 i 个结点的位置上，即插入到 a_{i-1} 与 a_i 之间。因此，必须首先找到 a_{i-1} 所在的结点 p ，然后生成一个数据域为 e 的新结点 q ， q 结点作为 p 的直接后继结点。



单链表中插入结点

□ 在第*i*个位置之前插入元素*e*:

```
Status ListInsert_L(LinkList &L, int i, ElemType e) {
```

```
    p = L; j = 0;
```

```
    while(p && j<i-1) {p = p->next; ++j;}
```

```
    if(!p || j>i-1) return ERROR;
```

```
    s = (LinkList)malloc(sizeof(LNode));
```

```
    s->data = e; s->next = p->next; p->next = s;
```

```
    return OK;
```

```
}
```

设链表的长度为*n*，合法的插入位置是 $1 \leq i \leq n$ 。

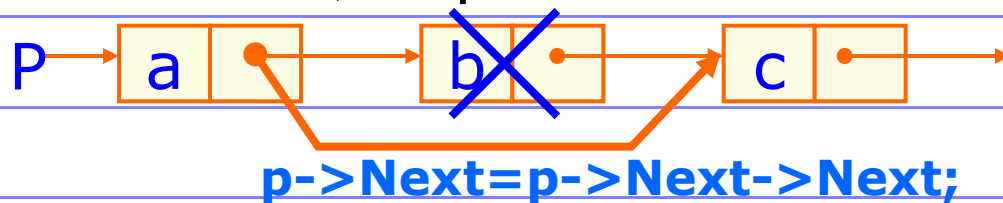
算法的时间主要耗费移动指针*p*上

□ 算法评价: $T(n) = O(n)$

□ 若给出的不是结点位置，而是结点数据，如何在给定的数据前/后插入数据*X*呢 ?

单链表中删除结点——按序号删除

□ 删除：单链表中删除b，设p指向a。算法描述如下：



```
Status ListDelete_L(LinkList &L, int i, ElemType &e) {  
    p = L; j = 0;  
    while(p->next && j<i-1) { p = p->next; ++j; }  
    if(!(p->next) || j>i-1) return ERROR;  
    q = p->next; p->next = q->next; e = q->data;  
    free(q);  
    return OK;  
}
```

□ 算法评价： $T(n) = O(n)$

单链表中删除结点——按值删除

删除单链表中值为key的第一个结点。与按值查找相类似，首先要查找值为key的结点是否存在？若存在，则删除；否则返回NULL。

```
void Delete_LinkList(LNode *L, int key)
/* 删除以L为头结点的单链表中值为key的第一个结点 */
{
    LNode *p=L, *q=L->next;
    while ( q!=NULL&& q->data!=key)
        { p=q; q=q->next; }
    if (q->data==key)
        { p->next=q->next; free(q); }
    else
        printf("所要删除的结点不存在!!\n");
}
```

□ 算法评价： $T(n) = O(n)$

单链表中删除结点——变形

从上面的讨论可以看出，链表上实现插入和删除运算，无需移动结点，仅需修改指针。解决了顺序表的插入或删除操作需要移动大量元素的问题。

变形之一：

删除单链表中值为`key`的所有结点。

与按值查找相类似，但比前面的算法更简单。

基本思想：从单链表的第一个结点开始，对每个结点进行检查，若结点的值为`key`，则删除之，然后检查下一个结点，直到所有的结点都检查。

单链表中删除结点——变形

算法描述

```
void Delete_LinkList_Node(LNode *L, int key)
```

```
/* 删除以L为头结点的单链表中值为key的第一个结点 */
```

```
{ LNode *p=L, *q=L->next;
```

```
while ( q!=NULL)
```

```
{ if (q->data==key)
```

```
{ p->next=q->next; free(q);
```

```
q=p->next; }
```

```
else
```

```
{ p=q; q=q->next; }
```

```
}
```

```
}
```

单链表中删除结点——变形

变形之二：

删除单链表中所有值重复的结点，使得所有结点的值都不相同。

与按值查找相类似，但比前面的算法更复杂。

基本思想：从单链表的第一个结点开始，对每个结点进行检查：检查链表中该结点的所有后继结点，只要有值和该结点的值相同，则删除之；然后检查下一个结点，直到所有的结点都检查。

单链表中删除结点——变形

算法描述

```
void Delete_Node_value(LNode *L)
/* 删除以L为头结点的单链表中所有值相同的结点 */
{
    LNode *p=L->next, *q, *ptr;
    while ( p!=NULL) /* 检查链表中所有结点 */
    {
        q=p, ptr=p->next;
        /* 检查结点p的所有后继结点ptr */
        while (ptr!=NULL)
        {
            if (ptr->data==p->data)
            {
                q->next=ptr->next; free(ptr);
                ptr=q->next;
            }
            else { q=ptr; ptr=ptr->next; }
        }
        p=p->next;
    }
}
```


链表的合并/连接

□ 把两个链表La和Lb合并

■ 若简单的合并两个链表，只需将第一个链表末尾元素的空指针改变指向第二个链表的头元素即可。

■ 若要合并两个有序表？

□ 特点：时间复杂性同一般顺序表，但空间需求减少。

链表的合并/连接

设有两个有序的单链表，它们的头指针分别是**La**、**Lb**，将它们合并为以**Lc**为头指针的有序链表。合并前的示意图如图所示。

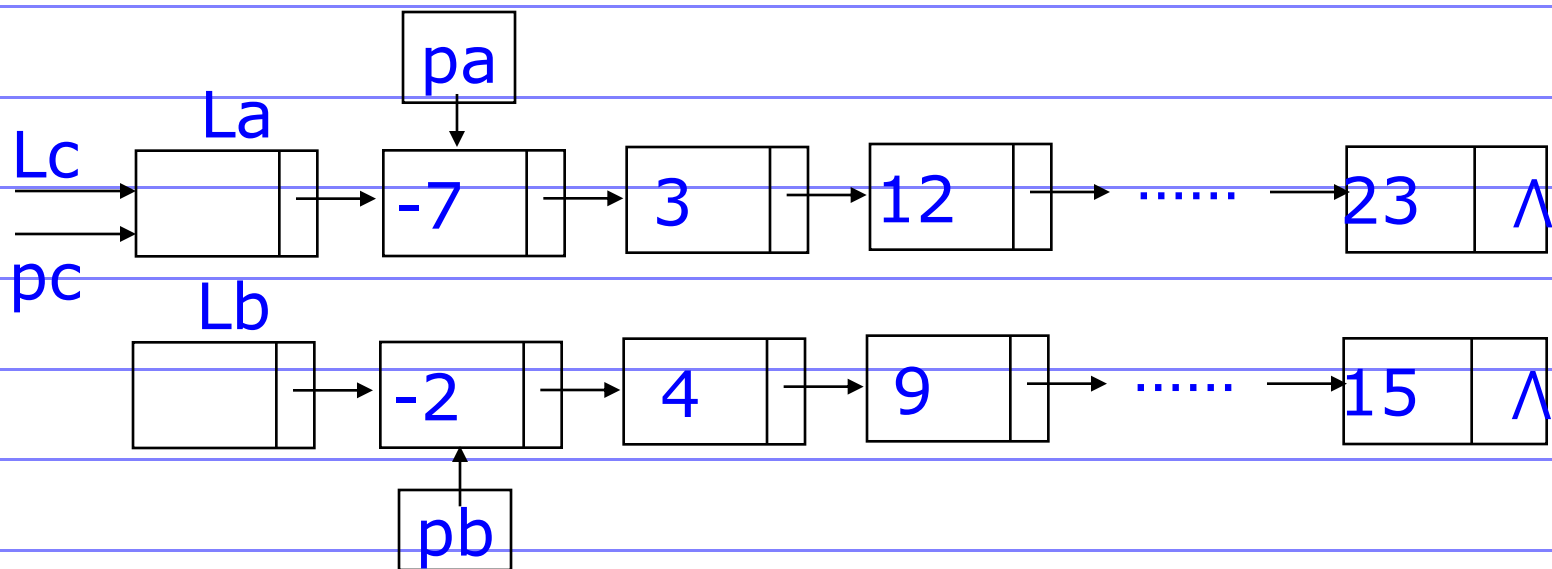
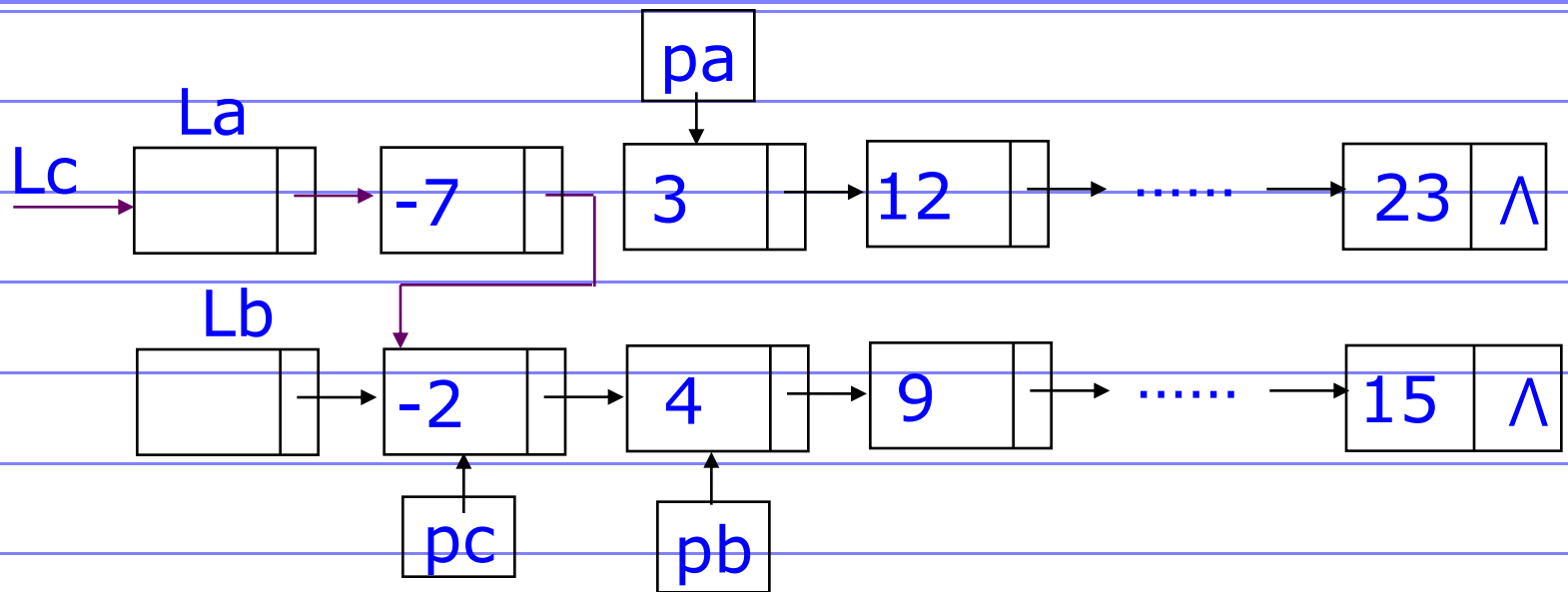


图 两个有序的单链表**La**，**Lb**的初始状态

链表的合并/连接



合并了值为-7，-2的结点后示意图

算法说明

算法中 **pa**，**pb** 分别是待考察的两个链表的当前结点，**pc** 是合并过程中合并的链表的最后一个结点。

链表的合并/连接

```
LNode *Merge_LinkList(LNode *La, LNode *Lb)
/* 合并以La, Lb为头结点的两个有序单链表 */
{
    LNode *Lc, *pa, *pb, *pc, *ptr;
    Lc=La; pc=La; pa=La->next; pb=Lb->next;
    while (pa!=NULL && pb!=NULL) {
        if (pa->data < pb->data) /* 将pa所指的结点合并, pa指向下一个结点 */
            { pc->next=pa; pc=pa; pa=pa->next; }
        if (pa->data > pb->data) /* 将pb所指的结点合并, pb指向下一个结点 */
            { pc->next=pb; pc=pb; pb=pb->next; }
        if (pa->data == pb->data) /* 将pa所指的结点合并, pb所指结点删除 */
            { pc->next=pa; pc=pa; pa=pa->next;
              ptr=pb; pb=pb->next; free(ptr); }
    }
    if (pa!=NULL) pc->next=pa;
    else pc->next=pb; /*将剩余的结点链上*/
    free(Lb);
    return(Lc);
}
```

若La, Lb两个链表的长度分别是m, n,
则链表合并的时间复杂度为 $O(m+n)$

单链表特点

- 它是一种动态结构，整个可用存储空间为多个链表共用
- 每个链表占用的空间不需预先分配，应需即时生成
 - 建立线性表的链式存储结构的过程就是一个动态生成链表的过程。即从“空表”的初始状态起，依次建立各元素的结点，并逐个插入到链表中。
- 指针占用额外存储空间
- 不能随机存取，查找速度慢？

静态链表

- 在一些无指针类型的高级语言中使用
- 通常使用一如下结构的数组

```
typedef struct {  
    ElemType    Data;  
    int         cur;  
} SLinkList[MAXSIZE];
```

称作静态链表

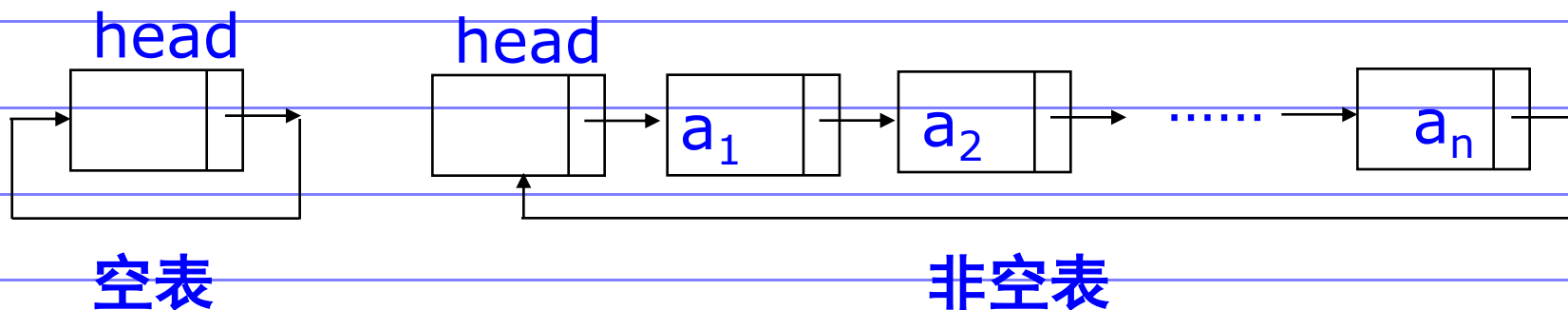
- 与顺序表用数组存储相比，
在插入或删除时无需移元素

0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	5
5	ZHOU	6
6	WU	7
7	ZHENG	8
8	WANG	0
9		
10		

0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	9
5	ZHOU	6
6	WU	8
7	ZHENG	8
8	WANG	0
9	BI	5
10		

循环链表(circular linked list)

- 循环链表(**Circular Linked List**): 是一种头尾相接的链表。其特点是最后一个结点的指针域指向链表的头结点, 整个链表的指针域链接成一个环。
- 从循环链表的任意一个结点出发都可以找到链表中的其它结点, 使得表处理更加方便灵活。



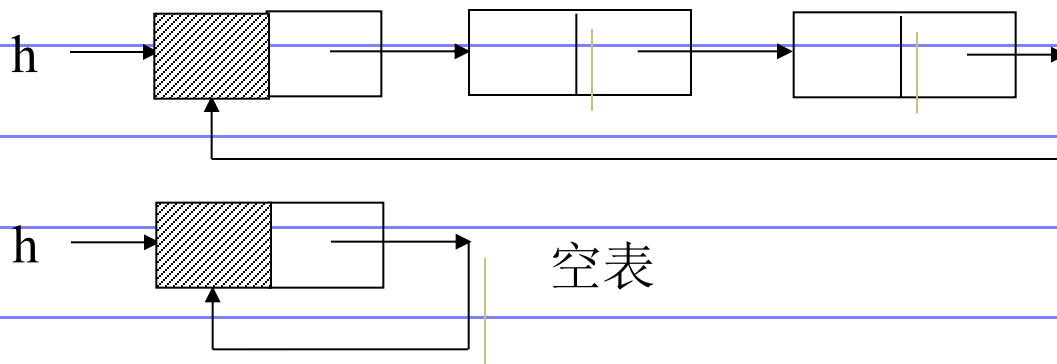
带头结点的单循环链表的示意图

循环链表(circular linked list)

□ 对于单循环链表，除链表的合并外，其它的操作和单线性链表基本上一致，仅仅需要在单线性链表操作算法基础上作以下简单修改：

■ 判断是否是空链表： **$\text{head} \rightarrow \text{next} == \text{head}$** ；

■ 判断是否是表尾结点： **$\text{p} \rightarrow \text{next} == \text{head}$** ；



双向链表(bi-linked list)

双向链表(**Double Linked List**) :指的是构成链表的每个结点中设立两个指针域：一个指向其直接前趋的指针域**prior**，一个指向其直接后继的指针域**next**。这样形成的链表中有两个方向不同的链，故称为双向链表。

和单链表类似，双向链表一般增加头指针也能使双链表上的某些运算变得方便。

将头结点和尾结点链接起来也能构成循环链表，并称之为双向循环链表。

双向链表是为了克服单链表的单向性的缺陷而引入的。

双向链表(bi-linked list)

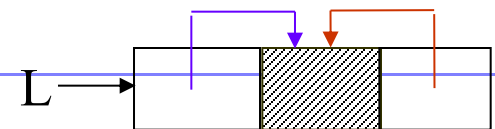
- ❑ 单链表具有单向性的缺点, 只能找后继, 不能找前驱。为查找一个结点总得从头结点处开始。双向链表是在结点中增加一个指向前驱的指针:

```
typedef struct DuLNode {  
    ElemType data;  
    struct DuLNode *prior,*next;  
}DuLNode; *DuLinkList;
```

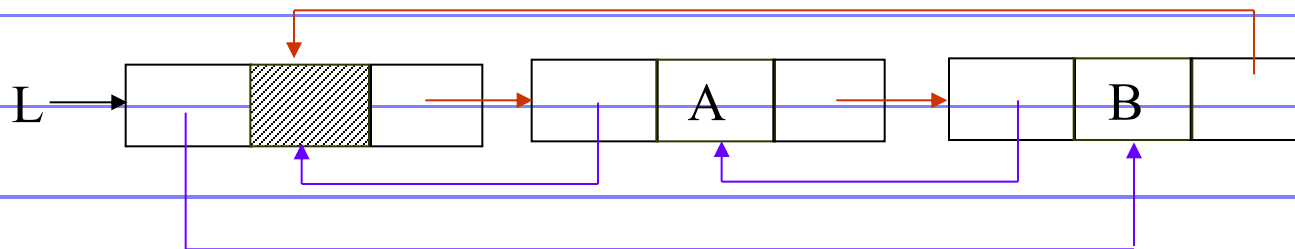
双向链表结点形式

prior	element	next
-------	---------	------

空双向循环链表

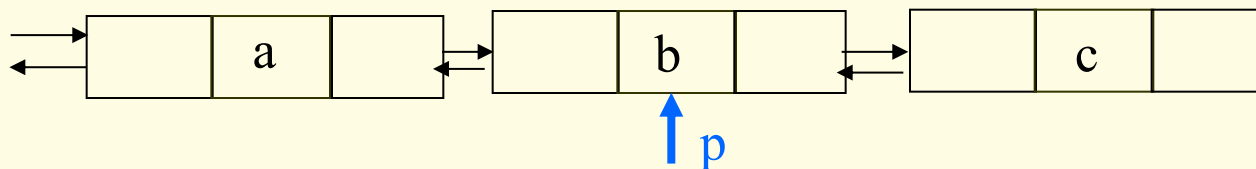


非空双向循环链表



双向链表特征

□ 双向链表结构具有对称性，设p指向双向链表中的某一结点，则其对称性可用下式描述：



$p \rightarrow \text{prior} \rightarrow \text{next} = p = p \rightarrow \text{next} \rightarrow \text{prior};$

结点p的存储位置存放在其直接前趋结点p->prior的直接后继指针域中，同时也存放在其直接后继结点p->next的直接前趋指针域中。

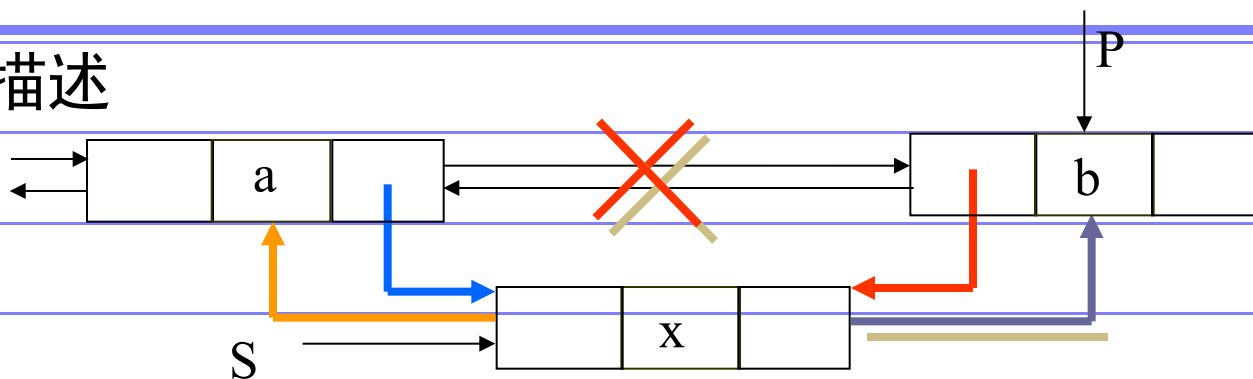
□ 双向链表的操作：

■ 插入

■ 删除

双向链表的插入操作

□ 算法描述

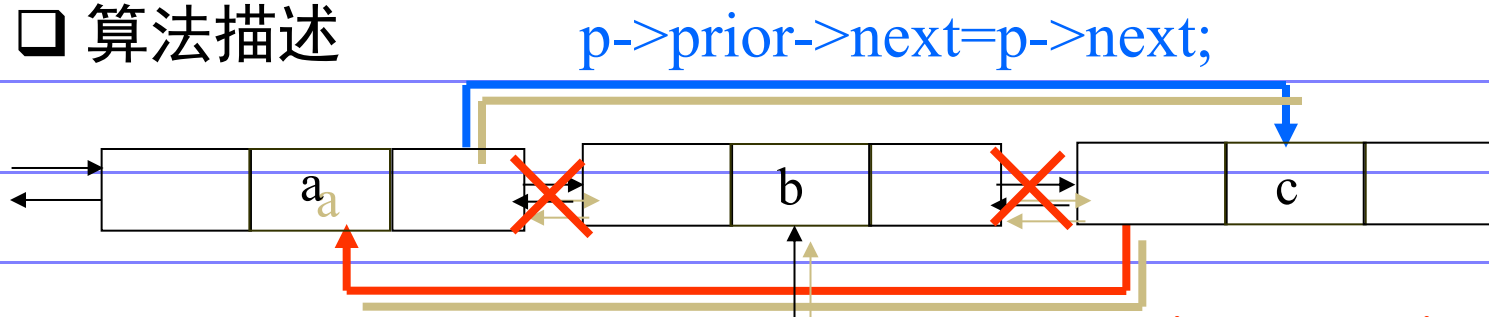


```
Status ListInsert_DuL(DuLinkList &L, int i, ElemType e) {  
    if(!(p = GetElemP_DuL(L, i))) return ERROR;  
    if(!(s = (DuLinkList)malloc(sizeof(DuLNode)))) return ERROR;  
    s->data = e;  
    s->prior = p->prior; p->prior->next = s;  
    s->next = p; p->prior = s;  
    return OK;  
}
```

□ 算法评价: $T(n)=O(n)$

双向链表的删除操作

❑ 算法描述



```
Status ListDelete_DuL(DuLinkList &L, int i, ElemType &e) {  
    if(!(p = GetElemP_DuL(L, i)))  
        return ERROR;  
    e = p_data;  
    p->prior->next = p->next;  
    p->next->prior = p->prior;  
    free(p); return OK;  
}
```

❑ 算法评价: $T(n) = O(n)$

❑ 与单链表的插入和删除操作不同的是, 在双向链表中插入和删除必须同时修改两个方向上的指针域的指向。

2.4 线性表的应用举例

□ 一元多项式的表示及相加

$$P_n(x) = P_0 + P_1x + P_2x^2 + \cdots + P_nx^n$$

可用线性表P表示 $P = (P_0, P_1, P_2, \cdots, P_n)$

但对S(x)这样的多项式浪费空间 $S(x) = 1 + 3x^{1000} + 2x^{20000}$

一般 $P_n(x) = P_1x^{e1} + P_2x^{e2} + \cdots + P_mx^{em}$

其中 $0 \leq e1 \leq e2 \leq \cdots \leq em$ (P_i 为非零系数)

用数据域含两个数据项的线性表表示

$$((P_1, e1), (P_2, e2), \cdots, (P_m, em))$$

其存储结构可以用顺序存储结构，也可以用单链表

2.4 线性表的应用举例

一元多项式的相加

不失一般性，设有两个一元多项式：

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n,$$

$$Q(x) = q_0 + q_1x + q_2x^2 + \dots + q_mx^m \quad (m < n)$$

$$R(x) = P(x) + Q(x)$$

$R(x)$ 由线性表 $R((p_0 + q_0), (p_1 + q_1), (p_2 + q_2), \dots, (p_m + q_m), \dots, p_n)$ 唯一表示。

顺序存储表示的相加

```
typedef struct
```

```
{ float coef; /*系数部分*/
```

```
  int expn; /*指数部分*/
```

```
} ElemType ;
```

线性表的定义

```
typedef struct
```

```
{ ElemType a[MAX_SIZE];
```

```
  int length ;
```

```
}Sqlist ;
```

用顺序表示的相加非常简单。访问第5项可直接访问

: **L.a[4].coef , L.a[4].expn**

单链表的结点定义

```
typedef struct node
```

```
{ float coef;
```

```
  int exp;
```

```
  struct node *next;
```

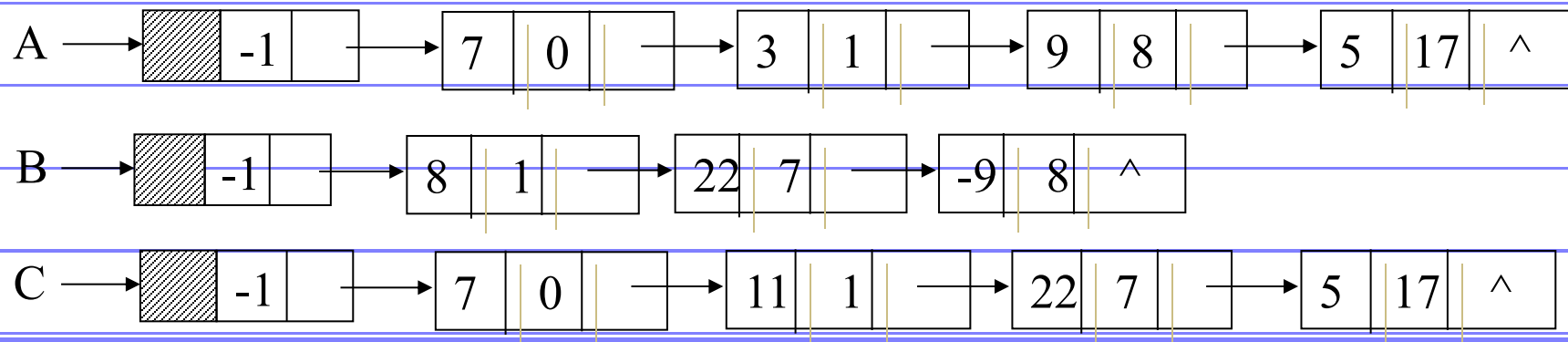
```
}Node;
```

coef	exp	next
------	-----	------

$$A(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$B(x) = 8x + 22x^7 - 9x^8$$

$$C(x) = A(x) + B(x) = 7 + 11x + 22x^7 + 5x^{17}$$



运算规则

□ 设 p, q 分别指向 A, B 中某一结点, p, q 初值是第一结点

比较

$p \rightarrow \text{exp} < q \rightarrow \text{exp}$: p 结点是和多项式中的一项
 p 后移, q 不动

$p \rightarrow \text{exp}$ 与 $q \rightarrow \text{exp}$ } $p \rightarrow \text{exp} > q \rightarrow \text{exp}$: q 结点是和多项式中的一项
将 q 插在 p 之前, q 后移, p 不动

$p \rightarrow \text{exp} = q \rightarrow \text{exp}$: 系数相加

- 0: 从 A 表中删去 p ,
释放 p, q , p, q 后移
- $\neq 0$: 修改 p 系数域,
释放 q , p, q 后移

链式存储表示的相加

当采用链式存储表示时，根据结点类型定义，凡是系数为0的项不在链表中出现，从而可以大大减少链表的长度。

一元多项式相加的实质是：

- 指数不同： 是链表的合并。

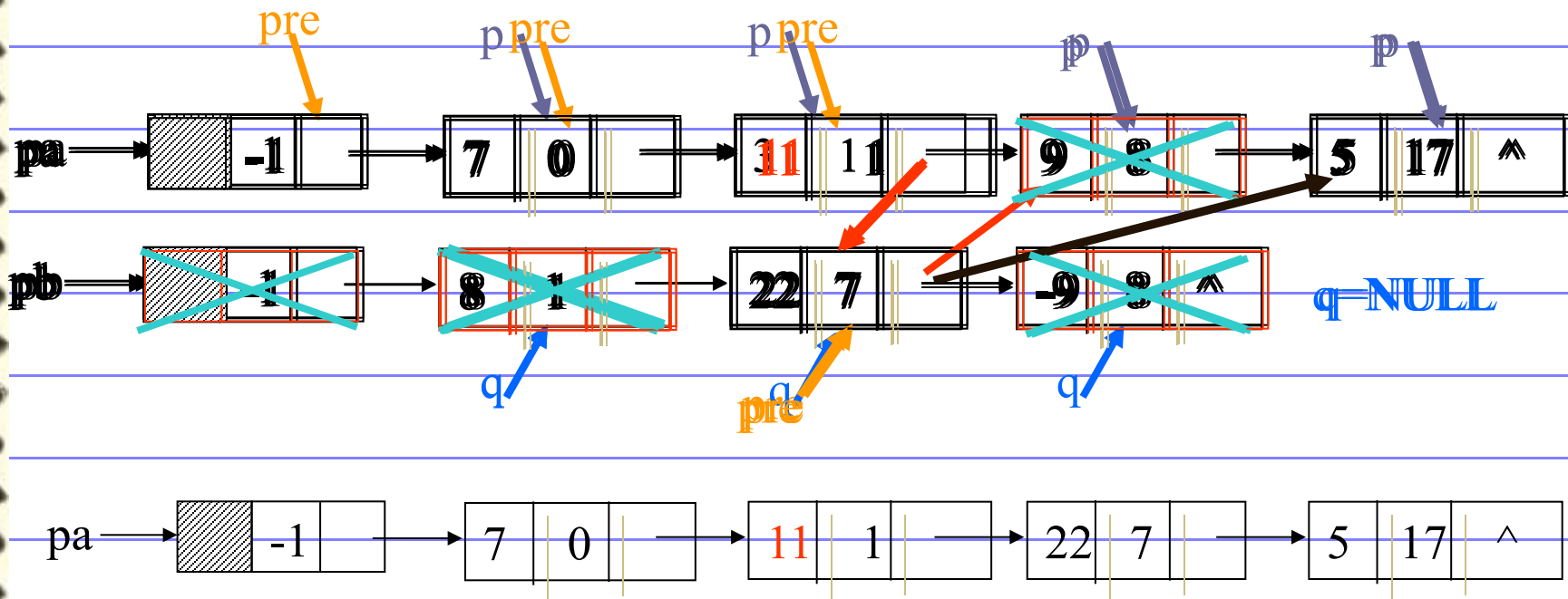
- 指数相同： 系数相加，和为0，去掉结点，和不为0，修改结点的系数域。

算法之一：

就在原来两个多项式链表的基础上进行相加，相加后原来两个多项式链表就不存在了。当然再要对原来两个多项式进行其它操作就不允许了。

算法描述

□ 算法描述



```
Ploy *add_ploy(ploy *La, ploy *Lb)
```

```
/* 将以La, Lb为头指针表示的一元多项式相加 */
```

```
{ ploy *Lc, *pc, *pa, *pb, *ptr; float x;
```

```
  Lc=pc=La; pa=La->next; pb=Lb->next;
```

```
  while (pa!=NULL&&pb!=NULL)
```

```
    { if (pa->expn<pb->expn)
```

```
      { pc->next=pa; pc=pa; pa=pa->next; }
```

```
      /* 将pa所指的结点合并, pa指向下一个结点 */
```

```
    if (pa->expn>pb->expn)
```

```
      { pc->next=pb; pc=pb; pb=pb->next; }
```

```
      /* 将pb所指的结点合并, pb指向下一个结点 */
```

```
    else
```

```
      { x=pa->coef+pb->coef;
```

```
        if (abs(x)<=1.0e-6)
```

```
          /* 如果系数和为0, 删除两个结点 */
```

```
          { ptr=pa; pa=pa->next; free(ptr);
```

```
            ptr=pb; pb=pb->next; free(ptr); }
```

```
        else /* 如果系数和不为0, 修改其中一个结点的系数域, 删除另一个结点 */
```

```
          { pc->next=pa; pa->coef=x;
```

```
            pc=pa; pa=pa->next;
```

```
            ptr=pb; pb=pb->next; free(pb);
```

```
          }
```

```
        }
```

```
      } /* end of while */
```

```
    if (pa==NULL) pc->next=pb;
```

```
    else pc->next=pa;
```

```
    return (Lc);
```

```
}
```


链式存储表示的相加

算法之二：

对两个多项式链表进行相加，生成一个新的相加后的结果多项式链表，原来两个多项式链表依然存在，不发生改变，如果要再对原来两个多项式进行其它操作也不影响。

Ploy *add_ploy(ploy *La, ploy *Lb) /* 将以La , Lb为头指针表示的一元多项式相加, 生成一个新的结果多项式 */

```
{    ploy *Lc , *pc , *pa , *pb , *p ; float x ;
    Lc=pc=(ploy *)malloc(sizeof(ploy)) ;
    pa=La->next ; pb=Lb->next ;
    while (pa!=NULL&&pb!=NULL)
        { if (pa->expn<pb->expn) { /* 生成的结点插入到结果链表的最后, pa指向下一个结点 */
            p=(ploy *)malloc(sizeof(ploy)) ; p->coef=pa->coef ; p->expn=pa->expn ;
            p->next=NULL ;
            pc->next=p ; pc=p ; pa=pa->next ; /* 生成一个新的结果结点并赋值 */
        }
        if (pa->expn>pb->expn) { /* 生成的结点插入到结果链表的最后, pb指向下一个结点 */
            p=(ploy *)malloc(sizeof(ploy)) ;
            p->coef=pb->coef ; p->expn=pb->expn ;
            p->next=NULL ;
            pc->next=p ; pc=p ; pb=pb->next ; /* 生成一个新的结果结点并赋值 */
        }
        if (pa->expn==pb->expn)
            { x=pa->coef+pb->coef ;
              if (abs(x)<=1.0e-6)
                  /* 系数和为0, pa, pb分别直接后继结点 */
                  { pa=pa->next ; pb=pb->next ; }
              else /* 若系数和不为0, 生成的结点插入到结果链表的最后, pa, pb分别直接后继结点 */
                  { p=(ploy *)malloc(sizeof(ploy)) ;
                    p->coef=x ; p->expn=pb->expn ;
                    p->next=NULL ;
                    /* 生成一个新的结果结点并赋值 */
                    pc->next=p ; pc=p ;
                    pa=pa->next ; pb=pb->next ;
                  }
            }
        } /* end of while */
    if (pb!=NULL)
        while(pb!=NULL)
            { p=(ploy *)malloc(sizeof(ploy)) ;
              p->coef=pb->coef ; p->expn=pb->expn ;
              p->next=NULL ;
              /* 生成一个新的结果结点并赋值 */
              pc->next=p ; pc=p ; pb=pb->next ;
            }
```

习题二

- 1 简述下列术语：线性表，顺序表，链表。
- 2 何时选用顺序表，何时选用链表作为线性表的存储结构合适？各自的主要优缺点是什么？
- 3 在顺序表中插入和删除一个结点平均需要移动多少个结点？具体的移动次数取决于哪两个因素？
- 4 链表所表示的元素是否有序？如有序，则有序性体现于何处？链表所表示的元素是否一定要在物理上是相邻的？有序表的有序性又如何理解？
- 5 设顺序表L是递增有序表，试写一算法，将x插入到L中并使L仍是递增有序表。

6 写一求单链表的结点数目ListLength(L)的算法。

7 写一算法将单链表中值重复的结点删除，使所得的结果链表中所有结点的值均不相同。

8 写一算法从一给定的向量A删除值在x到y($x \leq y$)之间的所有元素(注意：x和y是给定的参数，可以和表中的元素相同，也可以不同)。

数据结构题集（C语言版）：2.16, 2.21, 2.24, 2.25, 2.29, 2.31

习题一 易错题解析

1.8④ 设 n 为正整数。试确定下列各程序段中前置以记号@的语句的频度：

频度：该语句重复执行的次数

时间复杂度（渐进时间复杂度）：上述问题规模 n 的函数 $f(n)$ 的增长率相同的一个函数近似。

(5) for($i=1$; $i \leq n$; $i++$)

for($j=1$; $j \leq i$; $j++$)

for($k=1$; $k \leq j$; $k++$)

@ $x += 1$;

$1 + (1+2) + \dots + (1+2+\dots+n)$

$$\sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) = \frac{n(n+1)(n+2)}{6}$$

习题一 易错题解析

```
(6) i = 1; j = 0;  
    while(i+j <= n)  
    {  
        @ if(i>j) j++;  
        else i++;  
    }
```

n

习题一 易错题解析

```
(8) x = 91; y = 100;  
    while(y>0)  
    {  
        @ if(x>100) { x -= 10; y--; }  
        else x++ ;  
    }
```

11 * 100 = 1100

10次FALSE + 1次TRUE